Pushkaralu Reunite — Full Tech Stack Guide

Product: Lost-person reporting and reunification for **Godavari Pushkaralu 2027** (Rajamahendravaram)
Audience: Engineering, booth ops leads, reviewers
Covers: What we use, how the stack is wired, why these choices (and not others), and how to run the database under festival-scale load

---

1. Product in one page

Pushkaralu Reunite helps **pilgrims (USER)**, **volunteers**, and **police booth officers** find and reunite lost people during the festival.

Case lifecycle

```
MISSING → FOUND → MATCHED → REUNITED
```

Role	Primary client	Main jobs
USER	Expo app	Register family, wristband QR, report missing, track cases, language preference (en/te/hi/ta)
VOLUNTEER	Expo app	Create credentials / OTP login, register case from search, QR scan, alerts
POLICE	Expo app (+ Vite dashboard)	Booth allocation, cases, map, alerts, register case, QR scan, OTP / manual close

Privacy defaults (non-negotiable)

- Full Aadhaar **encrypted at rest**; UI shows masked form only
- Wristband QR encodes **UID only** (no personal payload)
- Face match gated by `FACE_MATCH_ENABLED` (default **off**)
- Soft-delete + retention windows; append-only `audit_log`

---

2. Monorepo map

Tooling: pnpm workspaces + Turborepo · **Node ≥ 20** · **pnpm 9.15**

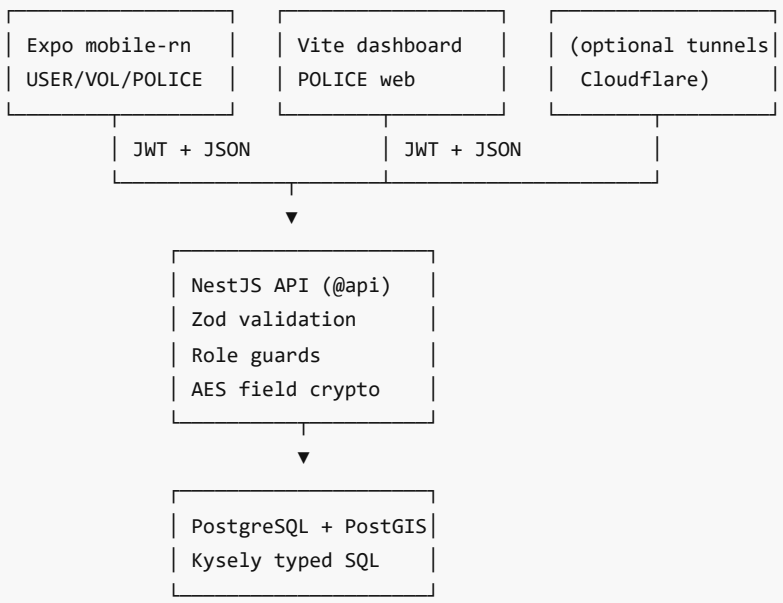
```
apps/
  api/          NestJS + TypeScript API (Kysely + PostGIS)
  dashboard/    React + Vite police booth dashboard
  mobile-rn/    Expo React Native (primary — all three roles)
  mobile/       Legacy Flutter (superseded by mobile-rn)
packages/
  shared/       Zod schemas, enums, case rules, UID, i18n
infra/
  docker-compose.yml   Optional PostGIS, Redis, MinIO, Adminer
docs/
  TECH_STACK.md
```

```
  Pushkaralu_Reunite_Tech_Stack.pdf
.github/workflows/ci.yml
```

Tool	Role in the stack
pnpm	Strict workspaces; one lockfile; fast CI installs
Turborepo	Parallel `build` / `lint` / `test` / `typecheck`
TypeScript	Shared types from API ↔ clients
ESLint + Prettier	Consistent style across packages

---

3. How the stack is used (runtime)



Shared contract: `@pushkaralu/shared` (Zod + enums + `assertCaseTransition`) so clients cannot invent illegal status moves.

Local data plane today: native PostgreSQL 17 + PostGIS (Docker Compose optional).
Demo sharing: Cloudflare quick tunnels for API + static Expo web export when needed.

---

4. Backend API (`apps/api`)

Piece	Technology	How we use it
Framework	**NestJS 10** + Express	Modules, DI, guards, pipes; `GET /health`
Queries	**Kysely** + `pg`	Type-safe SQL; no heavy ORM
DB	**PostgreSQL + PostGIS**	Cases, members, booths; `geography` + GIST
Validation	**Zod** + `ZodValidationPipe`	Same schemas as mobile/dashboard

Auth	**JWT** + Argon2	Family/police passwords; volunteer OTP
Crypto	**AES-256-GCM**	Phones, Aadhaar, address, photo keys
OTP / SMS	Mock / Twilio / MSG91	`SMS_PROVIDER`; mock prints `debugCode`
QR	`qrcode`	UID-only wristband payloads
Tests	Vitest	Unit + PostGIS integration in CI

Main modules

Auth · Members · Cases · FoundReports · Search · Volunteer · Booths · Notices · Crypto · Otp · RateLimit · Audit · Uid · Qr

Auth surfaces (current)

Endpoint area	Purpose
`/auth/register` + OTP	Pilgrim account
`/auth/login`	USER / POLICE password login
`/auth/police/register`	Create police credentials + booth link
`/auth/volunteer/request-otp` + verify	Volunteer login
`/auth/volunteer/register`	Create volunteer credentials + booth link
`/booths/directory`	Public booth list for create flows
`/booths/me`	Police allocated booth

Why NestJS (not Fastify-only, Django, Spring, Firebase Functions)

Choice	Why
NestJS	Structured modules for festival ops (auth, cases, audit); TypeScript end-to-end with Expo; strong guards/pipes for roles
Not Express alone	We want DI, modules, and consistent validation as the surface grows
Not Django/Rails	Team is TypeScript-first; shared Zod package would be awkward across languages
Not Firebase-only	Need PostGIS nearby search, audited handovers, and encrypted PII under our control
Not Spring	Heavier ops for a lean festival Phase-1; JS talent matches Expo

Why Kysely (not Prisma, TypeORM, Sequelize)

Choice	Why

Kysely	Explicit SQL + TypeScript types; PostGIS/`ST_*` stays natural
Not Prisma	Geospatial + custom retention/soft-delete patterns are cleaner in SQL
Not TypeORM	Avoids entity magic and migration surprises under pressure

---

5. Database: PostgreSQL + PostGIS

What lives in Postgres

`accounts` , `members` , `family_code_series` , `family_calls` , `cases` , `found_reports` , `handovers` , `volunteers` , `booths` , `notices` , `audit_log` , `consents` , `registration_drafts` , `otp_challenges`

Why PostgreSQL + PostGIS (not MongoDB, MySQL alone, DynamoDB)

Need	Why this DB
Relational cases + members + handovers	Strong consistency for reunification truth
Map / nearby volunteers	PostGIS `geography` + GIST
Encryption + audit	ACID + triggers (audit append-only)
Festival peak then quiet	Proven vertical + read-replica path

MongoDB would force us to reinvent relations and geo carefully. **MySQL** lacks first-class PostGIS. **DynamoDB** is poor for ad-hoc booth queries and spatial joins without extra services.

Migrations

Raw SQL registry (Windows-safe): `0001` init → `0002` auth/OTP → `0003` v3 UID/Aadhaar → `0004` Tamil language → `0005` `accounts.booth_id` → `0006` `booths.name_i18n` .

---

6. Handling large festival flow (database & API)

Festival traffic is **bursty**: peak at ghat hours, quiet overnight. Design for **read-heavy lists**, **write spikes on MISSING alerts**, and **hot booth desks**.

6.1 Data model habits (already in place)

1. **Soft-delete** (`deleted_at`) — reunited history stays for audit without polluting open lists.
2. **Status columns indexed in queries** — open cases filter `status <> 'REUNITED'` .
3. **Encrypted PII out of indexes** — search by UID / family code / hashed phone, not raw Aadhaar.
4. **Append-only audit** — forensics without updating history rows.
5. **Geography columns** — nearby volunteer alerts without scanning every case in app memory.

6.2 Indexing & query patterns (scale checklist)

Workload	Approach
Open cases by booth / status	Composite indexes on `(status, updated_at)` and booth linkage
Nearby MISSING for volunteers	PostGIS GIST on `cases.last_seen`; radius query with hard limit

Member lookup by UID	Unique index on `members.uid` (already)
Family list	Index `(account_id)` / `family_code`
Handover OTP	Short TTL rows; don't keep forever in hot tables
Notices	Small table; cache at edge or in Redis later

Rules: every list endpoint should be **paginated**; never `SELECT *` unbounded for festival-day dashboards.

6.3 Connection & write path

Layer	Practice
Pooling	`pg` pool sized to API replicas (e.g. PgBouncer in front for 10k+ clients)
Transactions	Keep short: allocate UID → insert member → commit
Idempotency	Report-missing / OTP verify should tolerate retries
Rate limits	Already on OTP/login; move counters to **Redis** under multi-instance API

6.4 Read scale (when booths / phones multiply)

1. **Primary** Postgres for writes (cases, handovers).
2. **Read replica** for police map polling, notices, booth directory.
3. **Redis** for: OTP rate limits, "seen alert" sets, hot case summaries (TTL seconds).
4. **Object storage (MinIO/S3)** for photos — DB stores encrypted key only (Compose already sketches MinIO).

6.5 Partitioning & archival (next step for multi-day festival)

Strategy	When
Partition `audit_log` / old `cases` by month or event_id	After Phase 1 if row counts explode
Archive `REUNITED` older than N days to cold storage	Post-event retention policy
Separate `event_id` column	If reusing the system for future Pushkaralu years

6.6 Observability under load

- API: request latency + error rate per route (`/cases` , `/search` , `/handover/*`)
- DB: slow query log, connection saturation, GIST scan times
- Alert when MISSING insert rate spikes (ops signal, not just infra)

6.7 What not to do at peak

- Don't run face-match as primary search
- Don't decrypt Aadhaar in list APIs
- Don't push full case history to every volunteer poll — use delta / since-timestamp
- Don't let CI or seed scripts point at production DB

---

7. Mobile app tech stack — Expo (`apps/mobile-rn`)

Primary field client for **USER / VOLUNTEER / POLICE**. Path: `apps/mobile-rn` (package `@pushkaralu/mobile`). `apps/mobile` (Flutter) is **legacy** and not the shipping app.

7.1 Libraries & platform

Technology	Version / notes	How used
Expo	~58	Dev client / Expo Go / web export
React Native	0.88	Native UI shell
React	19	UI + hooks
TypeScript	~5.9	App + shared types
React Navigation 7	native-stack + bottom-tabs	Role-based trees (user tabs, police tabs, volunteer home)
expo-camera	CameraView	Live wristband QR scan (+ manual UID fallback)
expo-location	—	Report-missing GPS, nearby alerts, booth directions
expo-av	—	Missing-alert sound (police / volunteer; mute on profile)
react-native-maps	—	Police / volunteer map surfaces
AsyncStorage	—	JWT session + pilgrim language preference
react-native-web	—	Browser / `expo export --platform web` demos
@pushkaralu/shared	workspace	Zod, roles, case rules, i18n keys

7.2 App features wired to this stack

Area	Behavior
Roles	One binary; login picks USER / VOLUNTEER / POLICE navigation
Language	**USER only** (en / te / hi / ta). Name + UID never translated
Booth names	API `nameI18n` JSON; UI picks label by pilgrim language
Cases	MISSING → FOUND → MATCHED → OTP or police manual close → REUNITED
QR	Camera scan + manual entry; payload is UID only
Alerts	Poll nearby MISSING; play sound on new case ids
Police	Cases filters (All / Missing / Found) + name/ID search; Booth tab; Find → Register case
Volunteer	Credential create + OTP; register case; QR; alerts

7.3 Why Expo RN (not Flutter-only, native Kotlin/Swift, pure PWA)

Choice	Why

Expo	Fast iteration with Expo Go / web export; camera, location, audio plugins
One codebase	Three roles share API client and navigation trees
Not Flutter-only	Flutter folder kept as legacy; Expo matched current shipping speed
Not two native apps	Cost and sync risk during festival build
Not PWA alone	Need reliable camera QR and alert UX on phones

Police/volunteer UIs stay English; pilgrims get en/te/hi/ta catalogs (including localized booth place-names).

7.4 Run it like a phone app

Prerequisites: Node ≥ 20, pnpm 9, API + Postgres running, **Expo Go** on the phone (or Android emulator).

```
# Terminal 1 — API
pnpm --filter @pushkaralu/api start:dev

# Terminal 2 — Expo (phone on same Wi-Fi; use your PC LAN IP)
# PowerShell:
#   $env:EXPO_PUBLIC_API_BASE = "http://YOUR_PC_LAN_IP:3000"
pnpm mobile
# Scan QR with Expo Go
```

Target	Command / note
Phone (Expo Go)	`pnpm mobile` + `EXPO_PUBLIC_API_BASE=http://<LAN-IP>:3000`
Android emulator	`EXPO_PUBLIC_API_BASE=http://10.0.2.2:3000` then `pnpm mobile:android`
Browser check	`pnpm mobile:web`
Static web demo	`npx expo export --platform web --output-dir dist-web` (set `EXPO_PUBLIC_API_BASE` first)

Default API assumptions without env: Android emulator `10.0.2.2:3000`, iOS/web `localhost:3000`.

---

8. Police dashboard (`apps/dashboard`)

Technology	Why
React 18 + Vite 6	Fast booth desktop UI
React Query	Cases / handover polling
MapLibre	Live map without Google billing lock-in
Proxy to API	Same JWT + shared types

Use when a large booth screen is better than a phone. Expo remains the field client.

---

9. Shared package (`packages/shared`)

Zod schemas · Role/CaseStatus enums · `assertCaseTransition` · UID helpers · i18n keys (`en` / `te` / `hi` / `ta`) · pilgrim search modes

Why a shared package (not OpenAPI-only or duplicate types): one source of truth prevents "mobile allows FOUND→REUNITED" bugs that break booth SOP.

---

10. Infrastructure & CI

Piece	Role
Docker Compose (optional)	PostGIS, Redis, MinIO, Adminer
GitHub Actions	`lint` → `build` → `test` with PostGIS service
Secrets	Local `.env` only (gitignored)

---

11. End-to-end reunification flow

1. Family registers → UIDs `K7M2-01` … + wristband QR.
2. User reports **MISSING**.
3. Volunteer/police **Register case** / QR scan → family call → case registered (**FOUND** path as designed).
4. Booth confirms → **MATCHED**.
5. Family OTP **or** police manual close (no signal) → **REUNITED**.
6. Audit trail retained; open lists hide reunited cases.

---

12. Stack at a glance

Layer	Technologies
Workspace	pnpm 9, Turborepo, TypeScript, ESLint, Prettier
API	NestJS 10, Kysely, pg, Zod, Argon2, JWT, AES-GCM, Vitest
Database	PostgreSQL + PostGIS (+ Redis/MinIO planned for scale)
Police web	React 18, Vite 6, React Query, MapLibre
Mobile app	Expo 58, RN 0.88, React 19, React Navigation 7, expo-camera/location/av, maps, AsyncStorage, shared i18n
Shared	Zod, enums, case rules, i18n
CI	GitHub Actions

---

13. Quick local commands

```
pnpm install
pnpm migrate:up
pnpm seed
pnpm --filter @pushkaralu/api start:dev
```

```
pnpm --filter @pushkaralu/dashboard dev
pnpm mobile                # Expo Go / Metro (set EXPO_PUBLIC_API_BASE for real phone)
# pnpm mobile:android      # emulator / device build
# pnpm mobile:web          # browser UI check
```

Demo login	Credentials
Pilgrim	`K7M2-01` / `pilgrim123`
Police	`BOOTH07-OFFICER` / `booth07pass`
Volunteer	`VOL-0214` + OTP (`debugCode` when SMS=mock)

See **§7** for the full mobile app tech stack and phone-run steps.

Source: `docs/TECH_STACK.md` · *PDF:* `docs/Pushkaralu_Reunite_Tech_Stack.pdf`